

Fundamentos da Plataforma NodeJS

Objetivo da Aula

Conhecer a metodologia de desenvolvimento de aplicativos com NodeJS, bem como a sintaxe JavaScript, e criar aplicativos simples na plataforma de desenvolvimento indicada.

Apresentação

Agora, iremos conhecer a plataforma NodeJS, como ela deve ser instalada e quais as ferramentas de desenvolvimento necessárias. Com o ambiente pronto, veremos como criar nossos primeiros aplicativos NodeJS.

Já que a programação nessa plataforma utiliza a linguagem JavaScript, será necessário compreender sua sintaxe e como ela evoluiu para absorver o uso de lambda no paradigma funcional. Iremos conhecer as formas para definição de variáveis e funções, controles de fluxo, orientação a objetos e uso do padrão arrow function.

Por fim, iremos analisar a arquitetura do NodeJS, compreendendo seu núcleo funcional, com especial atenção ao event loop e ao modelo single thread, que é garantido pela biblioteca LIBUV, implementada em linguagem C.

1. Criação de Aplicativos NodeJS

A plataforma NodeJS é um ambiente de código aberto e multiplataforma, para execução de aplicativos **JavaScript** e **Type Script** ao nível do servidor. Para começar a utilizá-lo, no ambiente Windows, devemos instalar o núcleo básico, que traz as ferramentas de linha de comando e os pacotes essenciais, acessando **<https://nodejs.org>**.



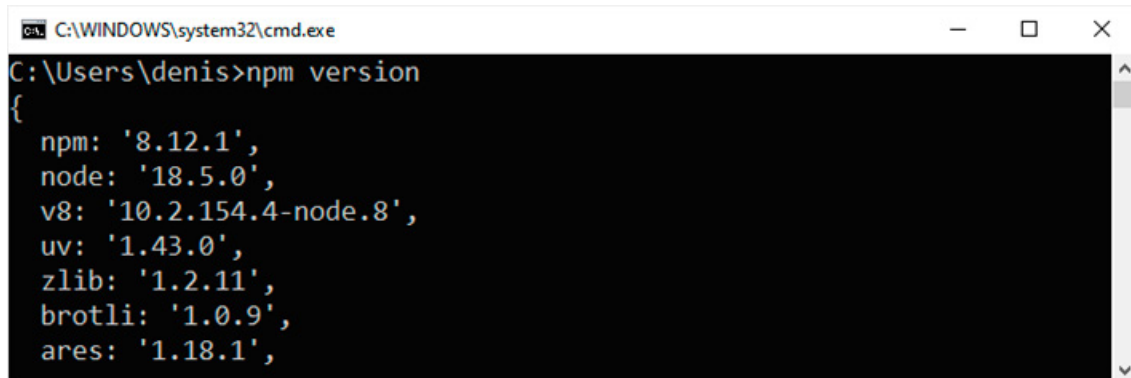
Destaque

Ao instalar o NodeJS, escolha a versão LTS, pois oferece suporte a longo prazo, sendo menos sujeita à obsolescência.

Com a plataforma instalada, você terá o comando **npm** disponibilizado para utilização via prompt ou no console do **Visual Studio Code**. Ele executa a maior parte das operações para gerenciamento de pacotes e configuração de projetos.

Para testar sua instalação, abra o prompt e execute o comando **npm version**, exibindo todas as informações do ambiente instalado.

Figura 1: Verificação da instalação do NodeJS



```
C:\WINDOWS\system32\cmd.exe
C:\Users\denis>npm version
{
  npm: '8.12.1',
  node: '18.5.0',
  v8: '10.2.154.4-node.8',
  uv: '1.43.0',
  zlib: '1.2.11',
  brotli: '1.0.9',
  ares: '1.18.1',
```

Fonte: Elaboração própria.

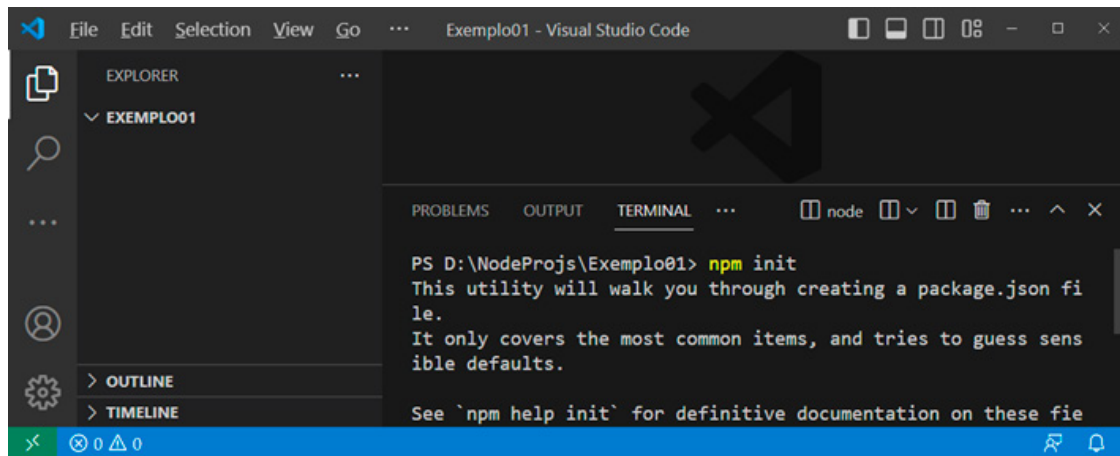
Antes de criar o seu primeiro aplicativo NodeJS, você deve instalar um editor adequado, sendo uma das melhores opções o Visual Studio Code, ou **VS Code**, disponível no endereço **<https://code.visualstudio.com/download>**. A instalação é muito simples e permite a escolha entre o tema padrão e o escuro para edição de códigos.

Diversas funcionalidades do VS Code trazem ganho de produtividade, como a complementação de código e a sintaxe realçada por cores. Além disso, oferece diversos pacotes complementares, para lidar com tipos específicos de projetos, e um console interno que permite executar o comando npm, sem a necessidade de interromper a execução do editor.

Você já pode criar seu primeiro projeto NodeJS, criando um diretório vazio, com o nome **Exemplo01**, e abrindo com o **VS Code**. Para abrir o diretório, utilize o menu **file**, seguido de **open folder**, no editor.

Com o diretório aberto no VS Code, abra o console interno, através do menu **terminal**, seguido de **new terminal**, e execute o comando **npm init** no diretório do projeto. Serão solicitadas algumas informações no processo de criação, mas você pode aceitar todos os valores padrão apertando a tecla **enter**. Ao final, você observará apenas a criação do arquivo **package.json**, contendo a configuração geral do projeto.

Figura 2: Configuração do projeto NodeJS

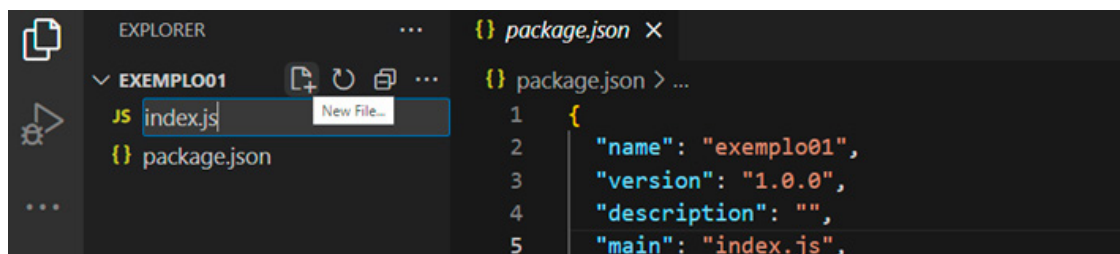


Fonte: Elaboração própria.

A interface do VS Code é bastante simples, com um painel lateral, no qual você pode navegar nos diretórios e arquivos do projeto, além de um editor de código-fonte na área central e os painéis de saída, entre eles o terminal, na área inferior.

Agora, você deverá criar o arquivo **index.js** no projeto, utilizando o ícone de ação **new file** no painel lateral. Note que esse foi o nome definido no atributo **main** do arquivo **package.json**.

Figura 3: Criação do arquivo index.js



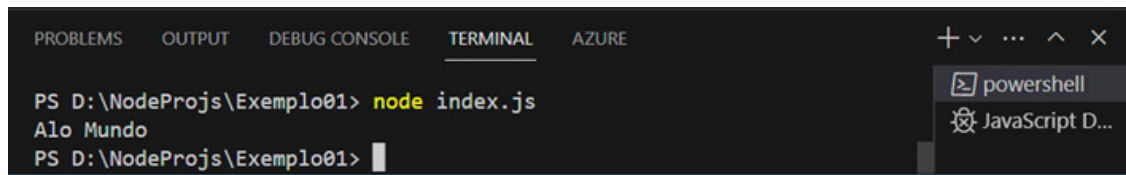
Fonte: Elaboração própria.

Altere o conteúdo do arquivo **index.js**, para o que é apresentado a seguir, e execute no console interno, através do comando **node index.js**.

```
console.log("Alo Mundo");
```

O comando **console.log**, em uma página HTML, ao executar no navegador envia informações para o painel de inspeção, mas quando é invocado ao nível do servidor exibe a informação no terminal de execução.

Figura 4: Execução do projeto Exemplo01



```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  AZURE
PS D:\NodeProjs\Exemplo01> node index.js
Alo Mundo
PS D:\NodeProjs\Exemplo01>
  
```

Fonte: Elaboração própria.

Note que criamos um aplicativo NodeJS de execução sequencial, em vez de definir um serviço HTTP. A plataforma fornece apenas um ambiente para execução fora do navegador, e serão as bibliotecas utilizadas que permitirão criar servidores para responder às requisições.

2. Sintaxe JavaScript

A linguagem JavaScript foi originalmente desenvolvida para ser executada no navegador, definindo a parte dinâmica de uma página HTML. Em 1996, a Netscape lançou o Netscape Enterprise Server 2.0, com a extensão LiveWire, que permitia executar JavaScript no servidor, sendo adotada com essa forma de execução desde então. Ela é interpretada, com tipagem fraca, e diferencia letras maiúsculas e minúsculas.



Você Sabia?

Atualmente, o JavaScript é baseado na especificação ECMA Script, ou ES, mantida pelo European Computer Manufacturers Association.

Nas versões iniciais da linguagem, uma variável podia ser definida através da palavra reservada **var**, mas, pelo padrão ECMA, você deve utilizar **let** para as variáveis e **const** para as constantes. A mudança é relevante, pois o uso de var permitia que ocorresse fuga de escopo para as variáveis.

Os tipos de valores nativos do JavaScript são o texto, o numérico e o booleano, além de permitir o valor **null** para elementos nulos. Note que null é diferente de **undefined**, também presente na linguagem para efetuar comparações, quando queremos verificar se determinado atributo existe para o objeto.

```
// Definição de variáveis com let
let a = 5;
```

```
let b = "Apenas uma frase";  
let c = true;
```

Por se tratar de uma linguagem derivada do C, JavaScript utiliza os mesmos operadores, com poucas diferenças devido à tipagem fraca. Ao trabalhar com C, uma divisão entre inteiros resulta em número inteiro, mas no JavaScript existe apenas o tipo numérico, sem distinção entre inteiro e ponto flutuante.

O uso de alguns operadores aritméticos pode ser observado a seguir.

```
// Exemplos de operações aritméticas  
let a = 5, b = 2;  
let c = a / b; // c vale 2.5  
let d = a % b; // d vale 1 (resto da divisão inteira)  
a += b; // a recebe o valor a + b  
a++; // a é incrementado de 1
```

Quando uma variável sofre uma operação sobre seu valor original, o operador pode ser explicitado à esquerda, o que simplifica a escrita de código, como você pode observar no exemplo anterior. Ao final do código, temos um exemplo do uso de pós-incremento, que adicionará uma unidade ao valor da variável.

Os operadores relacionais irão comparar dois valores, sempre retornando um booleano para a comparação efetuada. Você poderá observar alguns exemplos na listagem apresentada a seguir.

```
// Exemplos de operações relacionais  
let a = 5, b = 2;  
let c = a != b; // operador de diferença: c vale true  
let d = a == b; // operador de igualdade: d vale false  
let e = a > b; // operador maior que: e vale true  
let f = a <= b; // operador menor ou igual: f vale false
```

Temos, ainda, os operadores lógicos and, or e not, que combinam valores booleanos e serão utilizados nas instruções que envolvem condições.

```
// Exemplos de operações lógicas
let a = 5, b = 2, c = 3;
let b1 = (a > b) && (b > c); // operador and: T && F -> false
let b2 = (a > b) || (b > c); // operador or: T || F -> true
let b3 = !b2; // operador not:!T -> false
```

Para que você possa tomar decisões no fluxo de execução, deverá utilizar a instrução **if**, podendo adotar, também, o fluxo alternativo com **else**. Como não há uma palavra como then, o uso de parênteses é obrigatório, e, se a ação envolve mais de uma instrução, a delimitação do bloco com **chaves** é obrigatória.

```
let a = 5, b = 2, c = 3;
let maior;
if ((a>b)&&(a>c)) {
  console.log("A contém o maior valor");
  maior = a;
} else if((b>a)&&(b>c)){
  console.log("B contém o maior valor");
  maior = b;
} else {
  console.log("C contém o maior valor");
  maior = c;
}
```

Os principais controles de repetição da linguagem são **for** e **while**. Para o for, devemos considerar três seções, definindo a inicialização, o teste de finalização e o incremento, com muito mais flexibilidade que a estrutura para-faça tradicional.

```
// Impressão dos valores de 1 a 10
for(let a = 1; a <= 10; a++){
  console.log(a);
}
```

Uma grande mudança que ocorreu no JavaScript, ao adotar o padrão ECMA, foi na definição de funções. Na versão antiga, você só poderia definir uma função através da palavra **function**, mas, hoje, é possível utilizar o paradigma funcional, com base no operador de seta ou **arrow function**, e a substituição pelo novo formato está cada vez mais presente nos códigos em JavaScript, devido à sua praticidade e escrita concisa.

```
function somar_v1(a,b) {
  return a + b;
}
const somar_v2 = (a,b) => {
  return a + b;
}
const somar_v3 = (a,b) => a + b;
```

Na listagem, você pode observar três versões para a função de soma, sendo que a primeira utiliza function, e as duas seguintes o padrão arrow function, em que temos os parâmetros entre parênteses, bem como a seta e o corpo da função. Note que a terceira versão tem apenas a soma dos parâmetros, o que será permitido quando o corpo da função implementar apenas um retorno simples.

Você pode definir um vetor com a utilização de colchetes ou através da classe Array, e os vetores do JavaScript atuam como listas, com inclusão e exclusão de elementos de forma dinâmica.

```
let vetor1 = [1, 3, 5, 7]; // Vetor com os valores 1, 3, 5 e 7
let vetor2 = []; // Vetor vazio
let vetor3 = new Array(2, 4, 6); // Definição via classe Array
vetor2.push("ANA"); // Acréscimo de elemento no vetor
```

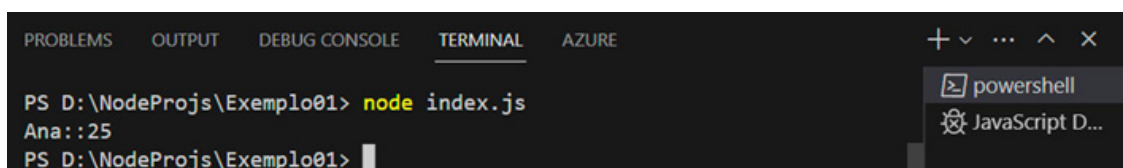
Outra mudança importante no padrão ECMA foi a inclusão de uma palavra para definição de classes, que, no caso, é **class**. Anteriormente, a palavra `function` era utilizada para definir classes no JavaScript.

```
class Pessoa{
  exibir = () => {
    console.log(`${this.nome}::${this.idade}`)
  }
  constructor (nome, idade){
    this.nome = nome;
    this.idade = idade;
  }
}

let p = new Pessoa("Ana",25);
p.exibir();
```

Na listagem, você poderá observar a definição de uma classe `Pessoa`, com um construtor que recebe os atributos `nome` e `idade`, além do operador **this**, que permite diferenciar o atributo da classe e o parâmetro do construtor. Note, também, a construção de texto dinâmico, delimitado com **crase**, em que cada trecho marcado com **cifrão** terá o valor substituído no texto final.

Figura 5: Exemplo com a classe `Pessoa`



Fonte: Elaboração própria.

3. Arquitetura Geral do NodeJS

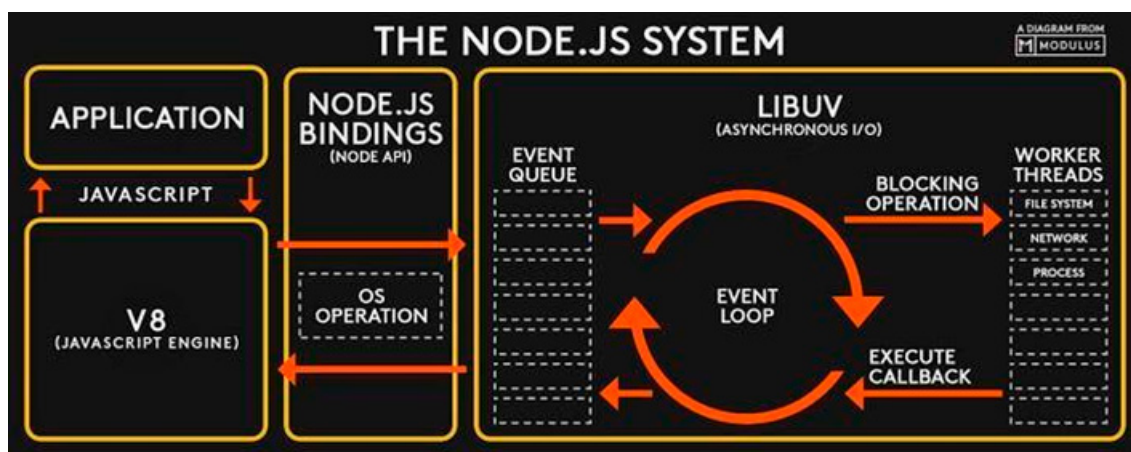
Operações de entrada e saída (I/O) são bastante custosas para qualquer sistema computacional. Se efetuadas de forma **síncrona**, elas bloqueiam o fluxo de execução do

programa, podendo até travar alguns dispositivos, fato pelo qual sistemas como o Android não permitem operações desse tipo fora de threads.

Essa também é uma preocupação primária no NodeJS, em que a arquitetura foi projetada para trabalhar com operações de I/O na forma **assíncrona**, ou seja, o fluxo de execução não é bloqueado, e os dados são recebidos em uma função de callback. Para garantir o comportamento assíncrono, foi utilizada a biblioteca **LIBUV**, escrita em linguagem C, baseada em um **pool de threads** e **sockets assíncronos**.

Para interpretar o JavaScript fora do navegador, é utilizado o **Chrome V8**, do Google, que oferece suporte a **ECMA Script** e **Web Assembly**, um mecanismo para empacotamento de conteúdo web na forma binária.

Figura 6: Modelo de execução do NodeJS



Fonte: <https://javascript.plainenglish.io/how-does-event-loop-work-in-node-js-d96729ae4a95>

O modelo de execução do NodeJS é baseado em **eventos**, organizados em uma **fila**, que são consumidos por um thread único, denominado **event loop**, o qual delega a execução das operações de I/O para o pool de threads. Ao final da execução, o thread que ficou responsável pela operação direciona o fluxo de informações para a função de **callback** que irá tratar o retorno.

Além do modelo baseado em eventos, o NodeJS utiliza um gerenciamento de pacotes associado diretamente ao **GitHub**. Através do comando **npm**, que é o acrônimo de "node package manager", você pode acrescentar bibliotecas ao seu projeto, e utilizando o comando **npv**, que vem de "node package extractor", pode executar comandos relacionados a pacotes globais.



Destaque

Todos os pacotes que forem adicionados ao projeto ficarão na pasta `node_modules`.

3.1. Thread Único

Com a utilização do event loop, o NodeJS segue o modelo de thread único ou single thread, no qual temos uma tarefa principal controlando todo o fluxo de execução e delegando para o pool de threads, ou worker threads, qualquer operação de I/O. A adoção desse modelo traz grandes vantagens em termos de uso de recursos, diminuindo o gasto com a troca de contexto dos modelos multithread tradicionais.

Uma desvantagem dessa abordagem é a de que impossibilita o uso de múltiplos processadores para o fluxo de execução principal, diminuindo a escalabilidade vertical. No entanto, facilita a implementação de microserviços em NodeJS com base no Docker.

3.2. Event Loop

Na prática, o event loop é um thread de execução infinita que tem como função consumir os eventos em uma fila. Quando você executa um programa no NodeJS, as operações assíncronas geram eventos, os quais são adicionados na fila, e, então, o event loop consome esses eventos, na ordem de entrada, delegando para os threads que irão tratar as operações de I/O.

A programação assíncrona é simples, bastando você emitir o evento e definir a função de callback utilizada no tratamento da recepção. Todo o restante ficará sob responsabilidade do NodeJS e de suas bibliotecas.

O código seguinte exemplifica o procedimento, efetuando uma chamada GET para o endereço do Google.

```
let $http = require("http");
const callback = (data) => {
  console.log(data);
};
```

```
const exibir_pagina = (url) => {  
  $http.get(url, callback);  
}  
exibir_pagina("http://www.google.com");
```

Ao executar o método **get** de **\$http**, é emitido o evento correto para efetuar o acesso à página, e uma função **callback** é associada no segundo parâmetro. O evento gerado é colocado na fila, consumido pelo **event loop**, direcionado para o pool de threads, e, quando chega a resposta da chamada HTTP, os dados são impressos no console, graças à função de callback.

Considerações Finais da Aula

Aqui, pudemos observar como funciona a plataforma NodeJS, a sua instalação e o gerenciamento de aplicativos e módulos via linha de comando. Vimos, também, que a utilização de uma ferramenta, como Visual Studio Code, traz um ambiente de grande produtividade para o desenvolvimento de aplicativos NodeJS.

Analizamos a sintaxe JavaScript, abordando todos os elementos básicos da programação na linguagem. Ainda, pudemos observar como a versão atual, gerenciada pelo ECMA, trouxe aspectos da programação funcional, com base no padrão arrow function, que será de ampla utilização em nossos projetos.

Ao final, pudemos analisar a arquitetura do NodeJS, que garante o tratamento assíncrono, com boa eficiência, através do modelo single thread e seu pool de threads para as operações mais demoradas. Vimos, também, como ocorre o uso de eventos na plataforma e seu gerenciamento pelo event loop.

Agora que entendemos o funcionamento do núcleo do NodeJS e vimos como utilizar os comandos npm e npx, já podemos criar nossos aplicativos, adotando como base a linguagem JavaScript.

Materiais Complementares

**Artigo**

Fundamentos NodeJS

2019, Titao Yamamoto Júnior.

O artigo aborda a estrutura interna do NodeJS, com seus componentes arquiteturais e modelo funcional no estilo Single Thread.

Link para acesso: <https://medium.com/@titaoyamamoto/trilha-nodejs-fundamentos-nodejs-6e32913e9f38> (acesso em 12 set. 2023.)



JavaScript Tutorial

2023, W3Schools (W3C).

O site traz diversas informações e exemplos acerca da sintaxe JavaScript, com a possibilidade de testar códigos on-line. Deve ser observada a navegação lateral para acesso aos tópicos.

Link para acesso: <https://www.w3schools.com/js/>. (acesso em 12 set. 2023.)

Referências

FLANAGAN, D. **JavaScript: o guia definitivo**. Porto Alegre: Bookman, 2014. Disponível em: <https://abre.ai/gKFE>. Acesso em: 12 set. 2023.

OLIVEIRA, C.; ZANETTI, H. **JavaScript descomplicado: programação para a Web, IoT e dispositivos móveis**. São Paulo: Érica, 2020. Disponível em: <https://abre.ai/gKFH>. Acesso em: 12 set. 2023.

OLIVEIRA, C.; ZANETTI, H. **Node.js: programe de forma rápida e prática**. São Paulo: Expressa, 2021. Disponível em: <https://abre.ai/gKFL>. Acesso em: 12 set. 2023.